Spécifications des Besoins

A. Spécifications Fonctionnelles (Orientées "User Stories")

Épic 1 : Gestion des Récoltes (Application Mobile- Agriculteur)

- **En tant qu'agriculteur**, je veux pouvoir créer une annonce avec une photo (prise depuis l'appareil), le type de produit, la quantité estimée et le prix, pour proposer ma récolte.
- **En tant qu'agriculteur**, je veux que mon application capte automatiquement ma position GPS lors de la création de l'annonce pour que les acheteurs connaissent ma zone.
- **En tant qu'agriculteur**, je veux pouvoir enregistrer une annonce même sans connexion internet (sauvegarde locale), afin de ne pas être bloqué dans mon champ.
- **En tant que système**, je dois synchroniser automatiquement les annonces stockées localement vers le serveur (MongoDB) dès que le smartphone capte un réseau (3G/4G/WiFi).

Épic 2 : Place de Marché et Achats (Plateforme Web- Acheteur Pro)

- **En tant qu'acheteur (restaurateur/supermarché)**, je veux filtrer les produits disponibles sur une carte interactive selon un rayon (ex: 50 km autour de mon établissement) pour privilégier les circuits courts.
- **En tant qu'acheteur**, je veux consulter la fiche détaillée d'un lot (photo, label bio/conventionnel, date de disponibilité) avant de l'ajouter à mon panier.
- **En tant qu'acheteur**, je veux pouvoir valider une commande de plusieurs lots et recevoir une confirmation instantanée.

Épic 3 : Gestion de la Transaction et Suivi

- **En tant qu'agriculteur**, je veux recevoir une notification Push sur mon téléphone dès qu'une commande est passée.
- **En tant qu'agriculteur**, je veux pouvoir accepter ou décliner une commande en fonction de mes capacités logistiques.
- **En tant qu'acheteur**, je veux un tableau de bord pour suivre le statut de mes commandes (En attente, Acceptée, En transit, Livrée).
- **En tant que système**, je dois générer une facture PDF automatique une fois la livraison confirmée par les deux parties.

B. Spécifications Non Fonctionnelles (Techniques)

Catégorie	Spécification Technique
Persistance Mobile	Utilisation de SQLite ou AsyncStorage dans React Native pour garantir le fonctionnement du mode hors-ligne sans crash de l'application.
Base de Données	Utilisation de MongoDB avec des index géospatiaux (2dsphere) pour optimiser les requêtes de recherche par rayon géographique.
Sécurité de l'API	Sécurisation de tous les endpoints Spring Boot via Spring Security et des tokens JWT (JSON Web Tokens). Mots de passe hashés (ex: Argon2).
Performance	Le temps de réponse de l'API pour une recherche cartographique complexe ne doit pas dépasser 2 secondes, même avec un volume de données important.
Compatibilité	La plateforme Web (React.js) doit être responsive. L'application mobile doit cibler a minima Android 8.0+ (très présent dans les zones agricoles).

C. Description Détaillée du Workflow du Projet (Processus Métier)

Ce workflow décrit le cycle de vie complet d'une donnée et d'une transaction sur Tellus, de la ferme jusqu'au restaurant.

Phase 1 : Saisie et Persistance de l'Offre (Mobile-> Backend)

1. **Saisie :** L'agriculteur ouvre son application React Native, prend une photo de sa récolte (ex: 200kg de tomates), saisit le prix et la date.
2. **Géolocalisation :** L'application récupère la latitude et la longitude via l'API de géolocalisation du téléphone.
3. **Traitement réseau :**
   - *Si réseau disponible :* L'application envoie immédiatement le payload JSON (données + coordonnées) et l'image (via Multipart) à l'API Spring Boot.
   - *Si aucun réseau :* L'application stocke les données dans la base de données locale du téléphone (SQLite/AsyncStorage). Un "Listener" réseau est activé.
4. **Synchronisation :** Dès que le réseau est détecté, la tâche de fond s'exécute, vide le cache local et pousse les données vers Spring Boot.
5. **Stockage :** Le backend Spring Boot valide les données et insère un nouveau document dans la collection Produits de MongoDB, incluant les coordonnées géospatiales.

Phase 2 : Découverte et Achat (Web-> Backend)

6. **Recherche :** Un acheteur se connecte sur l'application web React.js. Il définit un rayon de recherche de 30 km.
7. **Requête spatiale :** Le frontend web envoie les coordonnées de l'acheteur et le rayon au backend. Spring Boot exécute une requête géospatiale MongoDB ($near ou $geoWithin) et retourne les annonces pertinentes.
8. **Commande :** L'acheteur sélectionne les 200kg de tomates et valide sa commande.
9. **Mise à jour :** Spring Boot met à jour le statut du lot dans MongoDB (passe de "Disponible" à "Réservé") pour éviter les commandes en doublon.

Phase 3 : Validation et Logistique (Backend-> Mobile)

10. **Notification :** Le backend déclenche un événement qui via un service externe (ex: Firebase) envoie une notification Push à l'agriculteur.
11. **Validation :** L'agriculteur ouvre l'application, voit la commande et l'accepte. Le statut de la commande passe à "En préparation".
12. **Logistique (à définir) :** Le produit est expédié (soit l'agriculteur livre, soit l'acheteur récupère, soit un transporteur tiers est intégré).

Phase 4 : Clôture et Analytique

13. **Clôture :** L'acheteur confirme la réception de la marchandise sur la plateforme web.
14. **Facturation :** Spring Boot génère la facture (PDF), clôture la transaction et archive les données de la commande.
15. **Data :** Les données de la transaction (prix au kilo, date, produit) viennent alimenter le tableau de bord analytique de l'acheteur (généré par Recharts dans React) pour suivre l'évolution des prix du marché.